

ANÁLISIS DEL PROYECTO UniMatch

Resumen general:

UniMatch es una Single Page Application (frontend estático) consumiendo una API REST desarrollada en Spring Boot. En desarrollo usa H2 (archivo en data/), en producción está preparada para Oracle Cloud. La autenticación actual es un mecanismo simulado (token mock) y las contraseñas se almacenan en texto plano (MVP).

Raíz

- pom.xml: configuración Maven y dependencias (Spring Boot Web, Data JPA, Validation, H2, Oracle JDBC, Lombok, Spring Security Crypto opcional).
- proyectos.json: ejemplos de proyectos para UI.
- data/: contiene la base H2 en desarrollo (unimatch.mv.db).

Backend/ (controlador auxiliar)

- Backend/UniMatchController.java: endpoint demo en /api/v1/login que verifica dominio de email institucional.

Aplicación Spring Boot (src/main/java/com/unimatch)

- UniMatchApplication.java: clase principal con main().

Config (src/main/java/com/unimatch/config)

- CorsConfig.java: configura CORS permitiendo orígenes (*). Debe restringirse en producción.
- WebConfig.java: sirve recursos estáticos desde la raíz y redirige '/' a index.html.

Controladores (src/main/java/com/unimatch/controller)

- AuthController: registro y login (mock token "mock-jwt-token-{id}"). GET /auth/me devuelve usuario a partir del token mock. No hay hashing ni seguridad real.
- ChatController: endpoints para conversaciones y mensajes. Guarda imágenes en base64 en la BD (campo imagenBase64). Usa token mock para identificar usuario.
- ProjectController: CRUD básico para proyectos; al crear asigna director según token; permite finalizar proyectos y guardar resultados.
- SolicitudController: crear solicitudes a proyectos, listar enviadas/recibidas, responder (aceptar/rechazar). Aceptar incrementa participaciones del usuario.

Modelos (src/main/java/com/unimatch/model)

- Usuario: entidad principal con email único, password en claro, rol, carrera, lista de habilidades y contadores.
- Project: datos del proyecto, habilidades requeridas (ElementCollection), datos del director (redundantes para frontend) y campos de resultados.
- Solicitud: información de solicitud, estado y metadatos del solicitante.
- Mensaje: contenido, tipo, imagenBase64 (Lob), fecha y flags.
- Evaluacion: estrellas, etiquetas, reseña y referencias a proyecto/evaluador/evaluado.

Repositorios (src/main/java/com/unimatch/repository)

- Repositorios JPA para cada entidad con consultas por campos comunes (findByEmail, findByEstado, findByDirectorId, etc.).

Recursos estáticos (src/main/resources/static)

- index.html: SPA que carga los módulos JS.
- css/style.css: estilos y layout.
- js/ (frontend): app.js (router), firebase-config.js (API client y helpers), auth.js, projects.js, chat.js, profile.js. El frontend usa localStorage para token y usuario y configura API.BASE_URL hacia un endpoint ngrok.

Configuración (src/main/resources)

- application.properties: perfil por defecto 'h2', configuración de H2, JPA ddl-auto=update (dev), límites de subida y static-locations.
- application-oracle.properties: configuración preparada para Oracle (URL, credenciales, dialecto), con ddl-auto=none.

Flujo y observaciones importantes:

- El frontend espera `Authorization: Bearer mock-jwt-token-{id}` y many controllers parsean ese token manualmente.
- Contraseñas se guardan en texto plano; en producción debería usarse BCrypt y Spring Security.
- Almacenar imágenes en base64 en la BD puede causar problemas de rendimiento y tamaño. Recomendado: almacenamiento de objetos (S3, Blob Storage) y guardar solo URL.
- CORS y API.BASE_URL deben ajustarse a entornos reales (dominios permitidos, HTTPS, variables de entorno).

Recomendaciones rápidas:

- Implementar JWT real y middleware de seguridad (Spring Security).

- Usar BCrypt para passwords y validar entradas del usuario en backend.
- Mover almacenamiento de imágenes a un blob store y limitar tamaños en frontend/backend.
- Revisar y sanitizar datos que vienen del frontend antes de persistir.

Fin del análisis.